Use case: AL-9 On-Chain Settlement

Epic. An approved loan is recorded on a blockchain with a tamper-proof hash of the decision.

What changed in Sprint 2

- **The whole path from sign-off to a closed loan note now works on a local chain.** The underwriter's decision is saved (AL-166), the audit hash is written to the registry (AL-171, AL-172), the loan note is minted and the application becomes Funded (AL-66), and a funded loan can be closed, which closes the note on the chain as well (AL-293).
- **The audit hash now covers what was approved.** The record includes the loan type, the amount in whole cents and the term in whole months, so the hash on the chain also proves those weren't changed (AL-283).
- **An audit pack endpoint** gives the underwriter or the assigned broker the full record and recomputes the hash against the chain (AL-161). The portal's download button is Nikoloz's story (AL-38).
- **The borrower's app can show the loan note's state** because the application carries the note's token, transaction and chain name (AL-67). Ibrahima's story AL-39 is Done.
- **Still open:** saving every AI call as a document so the record can list them for real runs (AL-140, PR #177 in review), and the move to a public testnet (AL-183, Sprint 3).

Overview

Item	Detail
Epic	AL-9 On-Chain Settlement
Feature (from the brief)	Web: On-Chain Settlement and Decision Export (sign-off triggers minting; PDF/CSV audit packs). Core: Loan Tokenization Contract; Immutable Audit and Decision Registry; Execution Environment (Hardhat or Anvil locally, or Polygon Amoy or Arbitrum Sepolia). Mobile: lifecycle state of the digital loan note.
Owner	Solomon Sosanya (al-core: contracts and API). The portal stories are Nikoloz's and the mobile story is Ibrahima's.
Sprint	Sprint 1 (8 to 25 Oct 2026) delivered the loan note contract (AL-35). Sprint 2 (27 Oct to 15 Nov 2026) delivered the audit hash and registry (AL-36), the mint and the audit pack. Sprint 3 moves the contracts to a public testnet.

Goal

As a bank, I want approved loans recorded on a blockchain with a tamper-proof hash of the decision, so that anyone can later verify the loan and the reasons behind it.

Actors

- **Underwriter:** reviews an application in the web portal and signs it off.
- **al-core (the backend API):** records the decision and holds the minter wallet, the only wallet that can mint. The portal never holds a key.
- **The ledger:** a local Hardhat chain in Sprints 1 and 2, then a public testnet (Polygon Amoy or Arbitrum Sepolia) in Sprint 3.
- **Bank:** the holder of every loan note.
- **Borrower:** sees the lifecycle state of their loan note in the iOS app.
- **Auditor:** gets an audit pack and checks its hash against the one on the chain.

Preconditions

- The application has been through the AI committee review (status AI Deliberated) and the underwriter is signed in with the underwriter role.
- The loan note contract and the audit registry are deployed, and al-core's wallet holds the minter role.

Trigger: the underwriter approves an application after reviewing the application, the valuation and the AI committee's reasoning.

Main flow

1. The underwriter opens the loan file and reviews the application, the valuation and the AI committee's reasoning.
2. The underwriter approves, with a reason. The portal asks al-core to record the decision.
3. al-core checks the caller's role and that the status allows the move, then saves the decision (who, when, why) and sets the status to Underwriter Approved.
4. The underwriter asks for the mint. al-core builds the audit record (decision, risk tier, credit memo, loan type, amount, term, and the id and hash of every AI call) and takes its SHA-256: sorted keys, no spaces, timestamps in UTC.
5. al-core writes the hash to the audit registry. The registry accepts one hash per application and never replaces it.
6. al-core mints the loan note from its minter wallet. The note stores the principal in euro cents, the APR in basis points, the term in months, the loan type and two references: a hash of the application id and a salted hash of the borrower id. No personal data goes on the chain.
7. al-core saves the token id, contract address, transaction hashes, audit hash and chain name on the application, in one write, and sets the status to Funded. The portal and the borrower's app update from Firestore.
8. Later the underwriter closes the loan. al-core closes the note on the chain first, then saves the status Closed with the close time and transaction.
9. An auditor or the underwriter asks for the audit pack. It recomputes the hash and compares it with the saved one, the note's and the chain's. They match, so it says verified.

Alternative flows

- **Decline.** The underwriter declines with a reason. The decision is saved and the status becomes Declined. No note is minted and nothing goes to the chain. A declined application can still be closed, and that touches no chain either.
- **A retry after a cut-short mint.** If the chain accepted the mint but saving failed, the next mint finds the note already on the chain, checks its terms and finishes the save. Only one note ever exists for an application.

Exceptions

- The caller is not an underwriter, or the status doesn't allow the move: al-core refuses (403 or 409) and nothing changes.
- A second mint for the same application: the contract refuses it (`AlreadyMinted`) and al-core returns the existing note.
- A different hash for an application that already has one: the registry refuses it (`AlreadyStored`).
- The loan values are invalid, such as a zero principal, an APR above 100% or a term over 40 years: the contract refuses them and nothing is stored.

- The chain or the transaction fails: the application stays at Underwriter Approved and the mint can be retried. Nothing is marked Funded until the mint is confirmed.
- The record is changed after it was hashed: the audit pack says not verified.

Postconditions

- **Success:** the application is Funded, one loan note exists and its token id is saved, the audit hash is in the registry, and the audit pack says verified.
- **Failure or decline:** no note exists, and the status shows where it stopped.

Business rules

- Only al-core's wallet can mint or close a note, and a note can be closed once.
- One note per application.
- No personal data on the chain: only hashes and the loan terms.
- Notes are held by the bank and can't be transferred. A note is a record of the loan, not a tradable asset. Selling loans on is future work.
- The hash is the SHA-256 of the canonical record, so the same record always gives the same hash, in al-core and in al-ai.

Stories in this epic

Story	What	Owner	Sprint	Status on 8 Nov
AL-35	An approved loan is minted as a digital loan note	Solomon	Sprint 1	Done
AL-36	A SHA-256 hash of the credit memo, valuation and approval is stored on-chain	Solomon	Sprint 2	To Do in Jira until #177 merges; everything else in it is built
AL-151	An underwriter signs off an application and the decision is recorded	Nikolo z	Sprint 2	To Do (the backend, AL-166, is Done)
AL-37	Signing off from the portal triggers the loan note mint	Nikolo z	Sprint 2	To Do (the endpoint, AL-66, is Done)
AL-38	An underwriter downloads a PDF or CSV audit pack for a decision	Nikolo z	Sprint 2	To Do (the endpoint, AL-161, is Done)
AL-39	A borrower sees their loan note's on-chain lifecycle status	Ibrahima	Sprint 2	Done

What Sprint 2 delivered (Solomon)

- The shared audit hash function with test vectors, including a non-ASCII one (AL-253, AL-268).
- The `AuditRegistry` contract, written once per application, with Hardhat tests (AL-171). 47 Hardhat tests now pass across both contracts.
- The audit record and its registry write (AL-172), with the loan terms added (AL-283).
- The mint endpoint (AL-66), the loan note sync (AL-67) and the close endpoint (AL-293).
- The audit pack endpoint (AL-161).

- Checked on a local chain: mint, close, close again with no second transaction, the audit pack verified, then not verified after one character of the record was changed.